

A2 Level CS — Semester 1 Weeks 10–18: Hardware, System Software & AI | Victoria Park Academy

Victoria Park Academy

A2 Level Computer Science — Semester 1 Weeks 10 – 18

Hardware, System Software & Artificial Intelligence

Weeks 10 – 18: Hardware, System Software & Artificial Intelligence

Unit 3: Hardware (Weeks 10 – 13)

Unit 4: System Software (Weeks 14 – 16)

Unit 5: Artificial Intelligence (Weeks 17 – 18)

硬件、系统软件与人工智能 / Hardware, System Software & Artificial Intelligence

本单元涵盖逻辑电路、加法器、触发器、微处理器、操作系统内存管理、调度算法、中断处理以及人工智能基础。This unit covers logic circuits, adders, flip-flops, microprocessors, OS memory management, scheduling algorithms, interrupt handling, and AI fundamentals.

Week 10: Logic Circuits & Boolean Implementation

Week 10: Logic Circuits & Boolean Implementation

Learning Objectives

Design combinational logic circuits from Boolean expressions

Construct truth tables for multi-gate circuits

Implement circuits using NAND and NOR universal gates

Explain the relationship between Boolean algebra and physical hardware

Use Logisim to simulate and verify circuit designs

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): Recall Boolean laws from Week 1. "If I have the expression $A \cdot B + A \cdot C$, what circuit would I build?"

Direct Instruction (25 min): From expression to circuit: AND gates for products, OR gates for sums, NOT gates for complements. Show how to read a Boolean expression and draw the corresponding circuit diagram. Introduce circuit conventions: inputs on left, outputs on right, flow left-to-right.

Worked Example (20 min): Teacher designs circuit for $F = A \cdot B' + B \cdot C$. Step-by-step: identify AND terms, add NOT gates where needed, OR the results. Verify with truth table.

Guided Practice (25 min): Students design circuits for 3 expressions: Bronze (2-input), Silver (3-input with one NOT), Gold (4-input with multiple complements). Use Logisim on lab computers or paper templates.

Plenary (10 min): One student presents Gold solution. Class identifies any errors.

Lesson 2 (2 hours):

Starter (10 min): "Why do manufacturers prefer NAND gates over AND gates?" Discuss cost and universality.

Direct Instruction (25 min): Universal gates: NAND and NOR can implement any Boolean function. Demonstrate: NOT from NAND, AND from NAND, OR from NAND. Show De Morgan's equivalence: $A \cdot B = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$.

Worked Example (15 min): Convert $F = A \cdot B + C'$ to all-NAND implementation. Teacher draws original circuit, then replaces each gate with NAND equivalents.

Guided Practice (30 min): Students convert 3 circuits to all-NAND and all-NOR implementations. Verify using Logisim simulation.

Independent Practice (25 min): Design challenge: "Build a 2-to-1 multiplexer using only NAND gates." Students work individually or in pairs.

Plenary (10 min): Showcase multiplexer designs. Discuss why NAND is preferred in CMOS technology (fewer transistors).

Resources

Logisim installed on all lab computers

Circuit diagram templates (printed)

Physical logic gate kits for demonstration (if available)

Video: "Why NAND and NOR are Universal Gates" (Neso Academy)

Assessment Points

Formative: Circuit design worksheet (3 problems)

Formative: Logisim simulation files submitted

Homework: Design circuit for $F = (A + B) \cdot (C' + D)$ using only NAND gates; verify with truth table

Bronze: Draw circuits from simple 2-variable expressions; identify gate types.

Silver: Design multi-gate circuits from 3-variable expressions; convert to NAND/NOR implementations.

Gold: Design complex circuits from word problems; implement using only universal gates; explain CMOS advantages; design multiplexers.

Week 11: Half-Adders & Full-Adders

Week 11: Half-Adders & Full-Adders

Learning Objectives

Construct truth tables for half-adders and full-adders

Derive Boolean expressions for sum and carry outputs

Design half-adder and full-adder circuits using logic gates

Explain how full-adders are chained to create multi-bit adders

Calculate propagation delay in ripple-carry adders

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): Binary addition review: $101 + 110$. What are the outputs at each bit position?

Direct Instruction (25 min): Half-adder: adds two 1-bit numbers, produces sum and carry. Truth table, Boolean expressions ($\text{Sum} = A \oplus B$, $\text{Carry} = A \cdot B$). Circuit diagram with XOR and AND gates.

Worked Example (15 min): Teacher builds half-adder in Logisim, tests all input combinations, verifies truth table.

Guided Practice (20 min): Students construct half-adder truth table and circuit from scratch. Bronze: complete provided diagram; Silver: derive from truth table; Gold: implement using only NAND gates.

Direct Instruction (15 min): Full-adder: adds three 1-bit numbers (A, B, Carry-in). Truth table with 8 rows. Derive $\text{Sum} = A \oplus B \oplus \text{Cin}$, $\text{Cout} = A \cdot B + (A \oplus B) \cdot \text{Cin}$.

Plenary (5 min): Exit ticket: "Why can't a half-adder be used for all bit positions in multi-bit addition?"

Lesson 2 (2 hours):

Starter (10 min): Review full-adder expressions. "How do we add 4-bit numbers?"

Direct Instruction (20 min): Ripple-carry adder: chain full-adders. Carry-out of stage n becomes carry-in of stage $n+1$. 4-bit adder requires 4 full-adders. Show block diagram.

Worked Example (15 min): Trace 4-bit addition: $0110 + 0101$. Show carry propagation through each stage. Calculate total propagation delay: $4 \times \text{gate delay}$.

Guided Practice (25 min): Students trace 2 additions on provided 4-bit adder diagram. Calculate propagation delay assuming 2ns per gate. Bronze: one addition; Silver: two additions with delay; Gold: identify worst-case delay path.

Independent Practice (30 min): Research and explain: "What is a carry-lookahead adder? How does it improve on ripple-carry?" Students prepare a 3-minute explanation with diagram.

Plenary (10 min): 2 students present carry-lookahead explanations. Class discussion: when is the extra complexity worth it?

Resources

Logisim adder circuits (pre-built for demonstration)

Printed full-adder truth table templates

Ripple-carry adder diagram (A3 poster)

Article: "Carry-Lookahead Adders" (University notes)

Assessment Points

Summative: Quiz 3 (Week 11) — Hardware & Adders (20 marks)

Formative: Ripple-carry tracing worksheet

Homework: Design a 4-bit ripple-carry adder in Logisim; test with 3 input pairs; calculate propagation delay

Bronze: Complete half-adder truth table; identify sum and carry expressions; follow provided ripple-carry diagram.

Silver: Derive full-adder expressions; design full-adder circuit; trace multi-bit addition with carry propagation.

Gold: Implement full-adder with universal gates; calculate and optimise propagation delay; research and explain carry-lookahead; evaluate speed vs complexity trade-offs.

Week 12: Flip-Flops & Sequential Logic

Week 12: Flip-Flops & Sequential Logic

Learning Objectives

Explain the difference between combinational and sequential logic

Describe the operation of SR, D, and JK flip-flops

Construct truth tables (characteristic tables) for each flip-flop type

Explain the role of clock signals in synchronous circuits

Design simple sequential circuits: counters and shift registers

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): "What's the difference between a light switch and a toggle switch?" Introduce concept of state.

Direct Instruction (25 min): Combinational vs sequential logic: combinational output depends only on current inputs; sequential output depends on current inputs AND previous state. Introduce flip-flop as 1-bit memory element. SR flip-flop: Set-Reset with NOR/NAND implementation. Characteristic table, forbidden state ($S=R=1$ for NOR, $S=R=0$ for NAND).

Worked Example (15 min): Teacher traces SR flip-flop behaviour for input sequence: $S=1, R=0 \rightarrow S=0, R=0 \rightarrow S=0, R=1 \rightarrow S=0, R=0$. Show how state is maintained.

Guided Practice (25 min): Students complete SR flip-flop timing diagram for given input sequence. Bronze: complete provided diagram; Silver: draw from scratch; Gold: identify and explain the forbidden state condition.

Direct Instruction (15 min): D flip-flop: data input sampled on clock edge. Eliminates forbidden state. Positive-edge triggered vs negative-edge triggered. Show symbol with triangle (dynamic input).

Plenary (10 min): Compare SR and D flip-flops. "When would you use each?"

Lesson 2 (2 hours):

Starter (10 min): "How does a digital clock count seconds?" Introduce counters.

Direct Instruction (25 min): JK flip-flop: versatile with $J=K=1$ toggle mode. Characteristic table. T flip-flop (toggle) as special case. Binary counter: chain T flip-flops, each toggling when previous stage transitions from 1 to 0. Show 2-bit and 3-bit counter diagrams.

Worked Example (15 min): Teacher traces 3-bit binary counter through 8 clock cycles. Show output sequence: 000, 001, 010, 011, 100, 101, 110, 111, 000.

Guided Practice (25 min): Students design a 2-bit counter using D flip-flops. Derive next-state logic from state table. Bronze: complete provided design; Silver: design from state table; Gold: design modulo-5 counter (counts 0-4).

Independent Practice (25 min): Research: "What is a shift register? Name two applications." Students find real-world examples (serial communication, multiplication).

Plenary (10 min): Share shift register applications. Connect to USB, SATA, and other serial protocols.

Resources

Logisim flip-flop library (D, T, JK, SR)

Flip-flop timing diagram templates

Counter simulation files

Video: "Flip-Flops and Registers" (Neso Academy)

Assessment Points

Formative: Flip-flop timing diagram worksheet

Formative: Counter design task (marked with rubric)

Homework: Complete characteristic tables for SR, D, and JK flip-flops; draw timing diagram for given input sequence

Bronze: Complete provided timing diagrams; identify flip-flop types from symbols; follow counter sequences.

Silver: Draw timing diagrams from scratch; design simple counters; explain clock edge triggering; identify forbidden states.

Gold: Design non-standard counters (modulo-N); derive next-state logic from state tables; explain metastability; research and present on synchronous vs asynchronous counters.

Week 13: Microprocessors & Assembly Language

Week 13: Microprocessors & Assembly Language

Learning Objectives

Describe the main components of a CPU: ALU, CU, registers, buses

Explain the von Neumann architecture and stored program concept

Trace the fetch-decode-execute cycle step-by-step

Interpret and write simple assembly language instructions

Explain the role of the program counter, instruction register, and accumulator

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): "What happens inside your phone when you open an app?" Brainstorm CPU components.

Direct Instruction (30 min): Von Neumann architecture: CPU, memory, I/O connected by buses (address, data, control). CPU components: ALU (arithmetic/logic), CU (control unit), registers (PC, IR, ACC, MDR, MAR). Stored program concept: instructions and data in same memory.

Worked Example (20 min): Teacher traces fetch-decode-execute cycle for instruction LDA 5 (load accumulator with value at address 5). Show register contents at each step. Use diagram with arrows showing data flow.

Guided Practice (25 min): Students trace fetch-decode-execute for ADD 3 (add value at address 3 to accumulator). Bronze: follow provided template; Silver: trace independently; Gold: trace a 3-instruction sequence (LDA, ADD, STA).

Plenary (5 min): "Why is the PC incremented during the fetch phase?"

Lesson 2 (2 hours):

Starter (10 min): Review fetch-decode-execute. "How does the CPU know which operation to perform?"

Direct Instruction (25 min): Assembly language: mnemonic instructions corresponding to machine code. AQA-style instruction set: LDM, STO, ADD, SUB, JMP, CMP, JPE, JPN, OUT, END. Addressing modes: immediate (#) and direct. Demonstrate with simple program: add two numbers and output result.

Worked Example (15 min): Teacher writes and traces assembly program to calculate $15 + 7$. Show memory contents, register values, and output at each step using AQA assembler simulator.

Guided Practice (30 min): Students write assembly programs: Bronze — load two values, add, store; Silver — calculate average of three numbers; Gold — implement a loop that counts from 1 to 10.

Independent Practice (25 min): Debug challenge: given a buggy assembly program, identify and fix errors. Program should multiply a number by 4 using repeated addition.

Plenary (10 min): Discuss: "Why do we still learn assembly when high-level languages exist?" (embedded systems, performance, security)

Resources

AQA Assembly Language Simulator (<https://www.peterhigginson.co.uk/AQA/>)

CPU architecture diagram (A1 poster)

Fetch-decode-execute tracing template

Assembly language instruction reference card

Assessment Points

Formative: Fetch-decode-execute tracing worksheet

Formative: Assembly program marking (functionality and style)

Homework: Write assembly program to find largest of three numbers; trace execution showing all register values

Bronze: Name CPU components; follow provided fetch-decode-execute traces; write simple 3-line assembly programs.

Silver: Trace fetch-decode-execute independently; write assembly programs with branching; explain register roles.

Gold: Write complex assembly with loops and conditionals; trace multi-instruction sequences; compare CISC vs RISC; explain pipelining concept.

Week 14: OS Memory Management

Week 14: Operating System Memory Management

Learning Objectives

Explain the purpose of memory management in operating systems

Describe paging: fixed-size pages, page tables, logical to physical address translation

Describe segmentation: variable-sized segments, segment tables

Compare paging and segmentation advantages and disadvantages

Explain virtual memory and the role of page faults

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): "Why can't we just load every program into RAM at once?" Discuss memory limitations.

Direct Instruction (25 min): Memory management goals: allocation, protection, sharing, virtualisation.

Paging introduced: physical memory divided into frames, logical memory into pages. Page table maps page numbers to frame numbers. Address translation: split logical address into page number and offset.

Worked Example (20 min): Teacher demonstrates address translation. Page size = 1024 bytes. Logical address 2050. Page number = 2, offset = 2. Page table: page 2 → frame 5. Physical address = $5 \times 1024 + 2 = 5122$.

Guided Practice (25 min): Students complete 5 address translation exercises. Bronze: provided page table and simple addresses; Silver: construct page table from given mapping; Gold: calculate page table size and overhead.

Plenary (10 min): "What happens if a program tries to access a page not in memory?" Introduce page fault.

Lesson 2 (2 hours):

Starter (10 min): "Is dividing memory into fixed-size chunks always ideal?" Introduce segmentation.

Direct Instruction (25 min): Segmentation: logical program structure (code, data, stack, heap) mapped to variable-sized segments. Segment table contains base address and limit. Address translation: segment number + offset, with bounds checking.

Worked Example (15 min): Teacher demonstrates segmentation address translation with bounds checking. Show how segmentation prevents overflow (offset > limit → trap).

Guided Practice (25 min): Comparison table: paging vs segmentation. Students complete across 8 criteria (fragmentation, sharing, protection, table size, hardware support, etc.).

Independent Practice (30 min): Virtual memory research: "How does virtual memory allow a computer to run programs larger than physical RAM?" Students explain paging to disk (swap space), page fault handling, and thrashing.

Plenary (10 min): Class discussion: "Why do modern OSes use paging rather than pure segmentation?" (Linux uses paging; x86 segmentation is legacy)

Resources

Memory management diagram (A1 poster)

Address translation worksheets

Virtual memory animation (online)

Article: "Understanding the Linux Virtual Memory Manager"

Assessment Points

Summative: Quiz 4 (Week 14) — System Software (20 marks)

Formative: Address translation worksheet

Homework: Complete 8 address translations (4 paging, 4 segmentation); explain virtual memory with diagram

Bronze: Follow provided address translation steps; identify page number and offset; complete comparison table with guidance.

Silver: Perform address translation independently; construct page tables; explain internal vs external fragmentation; describe virtual memory operation.

Gold: Calculate page table overhead; analyse segmentation with multiple segments; explain page replacement algorithms (FIFO, LRU); discuss thrashing and working set model.

Week 15: Scheduling Algorithms

Week 15: CPU Scheduling Algorithms

Learning Objectives

Explain why CPU scheduling is necessary in multiprogramming systems

Describe and apply First-Come First-Served (FCFS) scheduling

Describe and apply Shortest Job First (SJF) scheduling

Describe and apply Round Robin (RR) scheduling with time quantum

Describe and apply Priority scheduling

Calculate waiting time, turnaround time, and response time for each algorithm

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): "If 5 students need to use one printer, how do you decide who goes first?" Analogies for scheduling.

Direct Instruction (25 min): Scheduling criteria: CPU utilisation, throughput, turnaround time, waiting time, response time. FCFS: simple queue, non-preemptive. SJF: minimises average waiting time, requires prediction. Preemptive vs non-preemptive.

Worked Example (20 min): Teacher calculates FCFS and SJF for 4 processes with given burst times. Show Gantt chart construction. Calculate average waiting time for each.

Guided Practice (25 min): Students complete Gantt charts and waiting time calculations for 3 scenarios. Bronze: FCFS only; Silver: FCFS and SJF; Gold: preemptive SJF (shortest remaining time first).

Plenary (10 min): "What are the disadvantages of SJF?" (starvation of long jobs, prediction difficulty)

Lesson 2 (2 hours):

Starter (10 min): "What if a new urgent job arrives while another is running?" Introduce preemption and Round Robin.

Direct Instruction (25 min): Round Robin: cyclic execution with fixed time quantum. Priority scheduling: highest priority runs first (preemptive or non-preemptive). Priority inversion problem. Multi-level feedback queues introduced briefly.

Worked Example (15 min): Teacher traces Round Robin with time quantum = 4 for 3 processes. Show context switches and partial execution.

Guided Practice (25 min): Students complete Round Robin and priority scheduling problems. Calculate turnaround time and response time. Bronze: Round Robin with 2 processes; Silver: Round Robin with 4 processes + priority; Gold: all algorithms with comparison of average metrics.

Independent Practice (25 min): Evaluation task: "A real-time system controls a medical device. Which scheduling algorithm is most appropriate? Justify your answer with criteria."

Plenary (10 min): Share evaluations. Discuss: no single algorithm is best for all scenarios.

Resources

Scheduling algorithm worksheets with Gantt chart templates

Online scheduler simulator

Comparison table template

Video: "CPU Scheduling Algorithms" (Neso Academy)

Assessment Points

Formative: Gantt chart and calculation worksheets

Formative: Evaluation task rubric assessment

Homework: Complete scheduling problems for all 4 algorithms; calculate average waiting time and turnaround time; recommend algorithm for given scenario

Bronze: Construct FCFS Gantt charts; calculate waiting time with guidance; identify basic scheduling criteria.

Silver: Apply all four algorithms; calculate waiting time, turnaround time, and response time; construct accurate Gantt charts.

Gold: Solve complex scheduling with arrival times and preemption; evaluate algorithms for specific scenarios; explain priority inversion and multi-level feedback queues; discuss real-time scheduling constraints.

Week 16: Interrupts & System Security

Week 16: Interrupts, Privilege Levels & System Security

Learning Objectives

Explain the purpose and types of interrupts (hardware, software, timer)

Describe the interrupt handling process: detection, saving state, executing handler, restoring state

Explain the role of the interrupt vector table

Describe privilege levels (user mode vs kernel mode) and protection rings

Explain how privilege levels protect system resources

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): "What happens when you press Ctrl+C while a program is running?" Introduce interrupts.

Direct Instruction (25 min): Interrupts: signals that pause normal execution. Types: hardware (I/O completion, keyboard), software (exceptions, system calls), timer (preemption). Interrupt handling: save PC and registers, load handler address, execute handler, restore state, return. Interrupt priority and

masking.

Worked Example (15 min): Teacher traces keyboard interrupt: user presses key, keyboard controller sends signal, CPU saves state, handler reads keycode, stores in buffer, restores state, resumes program.

Guided Practice (25 min): Students trace a timer interrupt in a multitasking system. Show how timer interrupt enables process switching. Bronze: follow provided trace; Silver: identify all saved/restored registers; Gold: explain interrupt nesting and priority.

Plenary (5 min): "Why are interrupts essential for multitasking?"

Lesson 2 (2 hours):

Starter (10 min): "Why can't a normal program directly access the hard disk?" Introduce privilege levels.

Direct Instruction (25 min): Protection rings: Ring 0 (kernel/supervisor mode) to Ring 3 (user mode). Privileged instructions only in Ring 0. System calls as controlled gateway from user to kernel mode. Memory protection: base and limit registers, page table protection bits.

Worked Example (15 min): Teacher traces a system call: program requests file read → trap to kernel → kernel validates request → performs I/O → returns result to user program. Show mode switch on diagram.

Guided Practice (25 min): Scenario analysis: "A virus tries to modify the page table directly." What prevents this? Students identify protection mechanisms at each level.

Independent Practice (25 min): Research: "Meltdown and Spectre — how do hardware vulnerabilities bypass privilege levels?" Students summarise in 5 bullet points.

Plenary (10 min): Discuss Meltdown/Spectre. Implications for system security and hardware design.

Resources

Interrupt handling diagram (A1 poster)

Protection rings diagram

System call tracing worksheet

Article: "Meltdown and Spectre Explained" (Ars Technica)

Assessment Points

Formative: Interrupt tracing worksheet

Formative: Privilege level scenario analysis

Homework: Draw interrupt handling sequence diagram; explain 3 ways privilege levels protect system resources

Bronze: Name interrupt types; follow provided interrupt handling sequence; identify user vs kernel mode.

Silver: Trace interrupt handling independently; explain interrupt vector table; describe system call mechanism; explain memory protection.

Gold: Explain interrupt nesting and priority; analyse protection ring architecture; discuss hardware vulnerabilities (Meltdown/Spectre); evaluate security implications of privilege level design.

Week 17: Expert Systems & Neural Networks

Week 17: Expert Systems & Neural Networks

Learning Objectives

Describe the components of an expert system: knowledge base, inference engine, user interface

Explain forward chaining and backward chaining inference strategies

Evaluate the advantages and limitations of expert systems

Describe the structure and operation of a biological neuron

Explain the perceptron model and its learning rule

Describe multi-layer neural networks and the role of hidden layers

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): "Can a computer diagnose an illness?" Show medical diagnosis app screenshot.

Direct Instruction (25 min): Expert system components: knowledge base (facts and rules), inference engine (applies rules), user interface (explanation facility). Rule format: IF condition THEN conclusion. Forward chaining: data-driven, starts with facts, derives conclusions. Backward chaining: goal-driven, starts with hypothesis, seeks supporting facts.

Worked Example (15 min): Teacher demonstrates forward chaining on animal identification expert system. Given facts: "has feathers," "can fly" → conclude "bird." Then demonstrates backward chaining: "Is it a bird?" → check feathers, beak, etc.

Guided Practice (25 min): Students complete forward and backward chaining exercises on a simple expert system (diagnosing computer faults). Bronze: follow provided rule set; Silver: add one new rule; Gold: design a 5-rule expert system for a domain of choice.

Plenary (5 min): "What are the limitations of expert systems?" (knowledge acquisition bottleneck, brittleness, no common sense)

Lesson 2 (2 hours):

Starter (10 min): "How does your brain recognise a face?" Introduce neural networks.

Direct Instruction (25 min): Biological neuron: dendrites (inputs), soma (processing), axon (output), synapse (connection). Artificial neuron (perceptron): weighted inputs, summation, activation function (step, sigmoid, ReLU). Perceptron learning rule: adjust weights based on error.

Worked Example (20 min): Teacher traces perceptron for AND gate. Initialise weights, calculate output for each input, adjust weights if error, repeat until converged.

Guided Practice (25 min): Students trace perceptron learning for OR gate. Bronze: provided weight update table; Silver: calculate updates independently; Gold: explain why XOR cannot be solved by single perceptron (linear separability).

Independent Practice (20 min): Multi-layer networks: input layer, hidden layer(s), output layer. Backpropagation introduced conceptually. Students explore TensorFlow Playground (online) to visualise hidden layers solving XOR.

Plenary (10 min): Discussion: "What can neural networks do that expert systems cannot?" (learning from data, generalisation, handling noise)

Resources

Expert system rule sets (printed)

TensorFlow Playground (<https://playground.tensorflow.org/>)

Perceptron tracing worksheets

Video: "How Neural Networks Learn" (3Blue1Brown)

Assessment Points

Summative: Quiz 5 (Week 17) — AI & Ethics (20 marks)

Formative: Expert system chaining exercises

Homework: Complete perceptron learning trace for AND gate; explain why multi-layer networks are needed for XOR; research one real-world expert system

Bronze: Identify expert system components; follow provided chaining examples; describe perceptron structure.

Silver: Apply forward and backward chaining independently; trace perceptron learning; explain activation functions; describe multi-layer networks.

Gold: Design original expert system; derive perceptron learning rule mathematically; explain backpropagation conceptually; evaluate expert systems vs neural networks for given problems.

Week 18: Machine Learning, Big Data & Ethics

Week 18: Machine Learning, Big Data & AI Ethics

Learning Objectives

Distinguish between supervised, unsupervised, and reinforcement learning

Describe the characteristics of big data: volume, velocity, variety, veracity

Explain how machine learning algorithms process big data

Identify ethical issues in AI: bias, privacy, accountability, transparency

Evaluate the societal impact of AI and big data technologies

Lesson Activities

Lesson 1 (2 hours):

Starter (10 min): "How does Netflix know what you want to watch?" Discuss recommendation systems.

Direct Instruction (25 min): Machine learning types: supervised (labelled data, classification/regression), unsupervised (unlabelled, clustering/association), reinforcement (agent, environment, rewards). Big data: 5 Vs — Volume (scale), Velocity (speed), Variety (types), Veracity (quality), Value (usefulness). Examples: social media, IoT sensors, financial transactions.

Worked Example (15 min): Teacher classifies 5 scenarios into ML types: spam filter (supervised), customer segmentation (unsupervised), game playing AI (reinforcement).

Guided Practice (25 min): Students classify 10 real-world applications into ML types and identify which "V" of big data is most relevant. Bronze: ML type only; Silver: ML type + relevant Vs; Gold: explain why a particular algorithm might be chosen.

Plenary (5 min): "What makes big data 'big'? Is it just the size?"

Lesson 2 (2 hours):

Starter (10 min): "Should an AI be allowed to decide who gets a loan?" Introduce ethics.

Direct Instruction (20 min): Ethical issues: algorithmic bias (training data reflects historical bias), privacy (surveillance, data collection), accountability (who is responsible for AI decisions?),

transparency (black box problem), job displacement, autonomous weapons. Framework for evaluation: utilitarian, deontological, virtue ethics.

Guided Practice (25 min): Case study analysis: "Facial recognition in public spaces." Students evaluate using ethical framework in groups. Bronze: identify one ethical issue; Silver: analyse from two perspectives; Gold: propose policy recommendation with justification.

Independent Practice (30 min): Students prepare a 2-minute position statement on one AI ethics topic of their choice. Use evidence from provided articles.

Plenary (15 min): Mini-debate: 3 students present positions, class questions. Teacher summarises key ethical principles.

Resources

Machine learning type classification cards

Big data case studies (Netflix, Amazon, healthcare)

AI ethics articles: "Weapons of Math Destruction" (O'Neil), EU AI Act summary

Video: "The Ethics of AI" (Royal Institution)

Assessment Points

Formative: ML classification worksheet

Formative: Ethics case study group analysis

Homework: Write 500-word evaluation: "Evaluate the ethical implications of using AI for university admissions."

Bronze: Identify ML types from descriptions; name one ethical issue; express a personal opinion on AI use.

Silver: Classify applications accurately; explain big data characteristics; analyse ethical issues using structured framework; consider multiple perspectives.

Gold: Evaluate ML algorithm choices for specific data characteristics; synthesise ethical arguments from multiple philosophical positions; propose evidence-based policy recommendations; discuss regulatory frameworks (EU AI Act, etc.).

Mock Examination — Week 18

End-of-Semester Mock Exam: Paper 1 Style

Examination Details:

Duration: 1 hour 45 minutes

Total Marks: 75

Format: Closed-book, bilingual glossary permitted

Coverage: All Semester 1 topics (Weeks 1–18)

Paper Structure:

Section	Marks	Time Guide	Topics
A: Short Answer	25	35 min	Boolean algebra, K-maps, floating-point, protocols
B: Structured Problem Solving	30	45 min	Logic circuits, adders, memory management, scheduling
C: Extended Response	20	35 min	AI ethics, system evaluation, design challenges

EAL Exam Support

Bilingual glossary provided (80+ terms)

Extra reading time: +15 minutes

Command words explained at start of each section

Diagrams provided for all hardware questions

Sentence starters for extended-response questions

Cross-Curricular Links

Subject	Connection	Activity
Mathematics	Boolean algebra, graph theory, binary arithmetic	Joint problem set on logic and proof
Physics	Electronics, semiconductor physics, signal propagation	Lab: measuring gate propagation delay
Philosophy	Ethics, consciousness, free will	Seminar: "Can machines think?" (Turing test discussion)
Business	Big data analytics, AI in marketing, automation	Case study: AI-driven business transformation
Politics	AI regulation, surveillance, digital rights	Debate: "Should facial recognition be banned in public?"

Homework Schedule (Weeks 10 – 18)

Week	Homework Task	Estimated Time
10	Design NAND-only circuit for given expression; verify with Logisim	60 min

Week	Homework Task	Estimated Time
11	Build 4-bit adder in Logisim; test with 3 pairs; calculate delay	60 min
12	Complete flip-flop characteristic tables; draw timing diagram	45 min
13	Write assembly program to find largest of 3 numbers; trace execution	60 min
14	Address translation exercises (paging and segmentation); explain virtual memory	60 min
15	Scheduling problems for all 4 algorithms; recommend for scenario	60 min
16	Draw interrupt sequence diagram; explain 3 privilege protection mechanisms	45 min
17	Perceptron learning trace for AND; explain XOR limitation; research expert system	60 min
18	500-word ethics evaluation: AI in university admissions	90 min

关键术语复习 / Key Term Revision:

Logic gate = 逻辑门 | Half-adder = 半加器 | Full-adder = 全加器 | Flip-flop = 触发器 | Microprocessor = 微处理器 | Assembly language = 汇编语言 | Fetch-decode-execute = 取指-译码-执行 | Paging = 分页 | Segmentation = 分段 | Virtual memory = 虚拟内存 | Scheduling = 调度 | Interrupt = 中断 | Expert system = 专家系统 | Neural network = 神经网络 | Machine learning = 机器学习 | Big data = 大数据 | Ethics = 伦理

Victoria Park Academy | A-Level Computer Science Department

Weeks 10 - 18 Scheme of Work v1.0 | Semester 1 2024 - 2025